Catatan: Refaktor Form Login dengan `createFormHook`

Dikerjakan: 2026-10-11. Pelengkap step.md. Rujukan aturan: ARCHITECTURE.md dan TECH-STACK.md.

> Catatan ini menjelaskan apa yang berubah, mengapa, dan bagaimana cara kerjanya. Baca berurutan; tandai bagian yang belum jelas untuk ditanyakan.

---

1. Masalah awal

`src/features/auth/components/login-form.tsx` panjangnya 116 baris dan melanggar dua aturan di ARCHITECTURE.md:

1. **Duplikasi.** Blok field `username` dan `password` hampir identik: sama-sama menghitung `isInvalid`, lalu merender `Field`, `FieldLabel`, input, dan `FieldError`. Aturan: pola yang muncul 2 kali atau lebih harus diekstrak.
2. **Terlalu besar.** Satu komponen mencampur form, tampilan field, dan state `showPassword`. Batas di dokumen sekitar 100 baris.

2. Konsep: `createFormHook`

Fitur TanStack Form untuk membuat "versi form sendiri" yang sudah mengenal komponen field buatan kita. Tiga bagian:

Bagian	Fungsi
Context	Jalur data tersembunyi. Komponen field mengambil objek `field` (nilai, error, handler) sendiri tanpa dikirim lewat props.
Komponen field	UI yang dipakai ulang (`TextField`, `PasswordField`).
`useAppForm`	Pengganti `useForm`. Mengenal komponen field, sehingga bisa dipanggil sebagai `field.TextField`.

3. Struktur file sesudah refaktor

```
src/features/auth/
  components/
    login-form.tsx            # tinggal merangkai (56 baris)
    fields/
      text-field.tsx          # baru
      password-field.tsx      # baru
  hooks/
    form-context.ts           # baru
    use-app-form.ts           # baru
```

```
  schemas/
    auth.ts                    # tidak berubah
```

4. File baru

4.1 `hooks/form-context.ts`

```ts
export const { fieldContext, formContext, useFieldContext } =
  createFormHookContexts()
```

- `createFormHookContexts()` menghasilkan dua context (field dan form) dan hook `useFieldContext`.
- Dipisah dari `use-app-form.ts` untuk **menghindari impor melingkar**: `use-app-form.ts` mengimpor komponen field, dan komponen field mengimpor context. Jika context ada di `use-app-form.ts`, keduanya saling mengimpor.

4.2 `components/fields/text-field.tsx`

Isi blok `username` lama, dipindah dan dibuat generik.

Sebelum	Sesudah
`field` dari parameter render-prop `form.Field`	`field` dari `useFieldContext<string>()`
Label dan placeholder tertulis tetap	`label` menjadi props; sisanya diteruskan lewat `...inputProps`
`isInvalid` dihitung di setiap field	Dihitung sekali di sini

Tipe props:

```ts
type TextFieldProps = Omit<
  React.ComponentProps<typeof Input>,
  "id" | "name" | "value" | "onChange" | "onBlur"
> & { label: string }
```

- Menerima semua props `Input` (placeholder, autoComplete, dll.).
- Lima props yang dikendalikan form **dilarang** dikirim dari luar, supaya pemanggil tidak merusak sambungan ke form.
- Urutan JSX sengaja `{...inputProps}` dulu, baru `id`, `name`, `value`, dst. Props milik form harus menang jika bentrok.

4.3 `components/fields/password-field.tsx`

Sama seperti `TextField`, tetapi memakai `InputGroup` dan tombol mata.

- `showPassword` (`useState`) pindah ke sini sebagai **state lokal komponen**. Ini bukan state form, jadi tidak melanggar aturan "form wajib TanStack Form".
- `type` ikut dilarang di props karena komponen yang mengatur `text` atau `password`.
- `aria-label` tombol mata dibuat dari `label`: `` `Sembunyikan ${label}` `` / `` `Tampilkan ${label}` ``. Hasilnya tetap "Tampilkan Kata Sandi" / "Sembunyikan Kata Sandi".
- Tombol mata tetap `type="button"` supaya tidak men-submit form.

4.4 `hooks/use-app-form.ts`

```
export const { useAppForm } = createFormHook({
  fieldContext,
  formContext,
  fieldComponents: { TextField, PasswordField },
  formComponents: {},
})
```

- `fieldComponents`: komponen yang nanti muncul sebagai `field.TextField` dan `field.PasswordField`.
- `formComponents` dikosongkan karena belum ada komponen tingkat form (mis. tombol submit khusus). Tombol Masuk tetap `Button` biasa.

5. File yang diubah: `components/login-form.tsx`

Bagian	Sebelum	Sesudah
Impor	`useForm`, `React`, `Eye`/`EyeOff`, `Input`, `InputGroup*`, `Field*`	Hanya `Button`, `FieldGroup`, `useAppForm`, `loginSchema`
State	`useState` untuk `showPassword`	Dihapus (pindah ke `PasswordField`)
Hook form	`useForm({...})`	`useAppForm({...})`, argumen sama
Field	`form.Field` + ±25 baris JSX per field	`form.AppField` + 1 komponen per field

Contoh pemakaian sekarang:

```
<form.AppField name="username">
  {(field) => (
    <field.TextField
      label="Nama Pengguna"
      placeholder="Nama Pengguna"
      autoComplete="username"
    />
  )}
</form.AppField>
```

Tidak berubah: `onSubmit` dengan `preventDefault` + `stopPropagation`, `FieldGroup`, tombol Masuk, tautan "Lupa Kata Sandi?", `console.log`, dan `<a href="#">`.

Hasil: **116 baris menjadi 56 baris.**

6. Perilaku di layar

Seharusnya sama seperti sebelumnya: validasi saat submit lewat `loginSchema`, error tampil setelah field disentuh, tombol mata bekerja sama. Kesimpulan ini dari typecheck dan pembacaan kode; **belum dicoba di browser**.

7. Perubahan konfigurasi (terpisah dari refaktor)

- `.prettierignore`: ditambah `src/routeTree.gen.ts` (file hasil generate, tidak boleh diedit manual).
- `eslint.config.js`: `src/components/ui/field.tsx` ditambah ke `ignores`. Komponen shadcn bawaan itu memicu error `no-unnecessary-condition` di baris 195; aturan melarang mengubah isi `components/ui` tanpa alasan.
    - Konsekuensi: error lain di file itu ikut tak terdeteksi.
    - Jika komponen diperbarui lewat CLI shadcn, file akan tertimpa; cek lagi apakah pengecualian masih perlu.

8. Verifikasi

`npm run typecheck`, `npm run lint`, dan `npm run check` lolos semua.

9. Sisa pekerjaan

- [ ] Ganti `console.log` di `onSubmit` dengan server function.
- [ ] Ganti `<a href="#">` dengan `Link` TanStack Router.
- [ ] Coba form di browser (validasi kosong, tombol mata, submit).
- [ ] Jika fitur lain butuh field yang sama, pindahkan `TextField` dan `PasswordField` ke `src/components/form/` (fitur tidak boleh mengimpor dari fitur lain).

10. Pertanyaan untuk dibahas

Tuliskan di sini bagian yang belum dipahami.